

EXPERIMENT-09(HDOC)

C Programming

Question 1: Convert Time in Seconds to Days, Hours, Minutes and Seconds

Step 1: Understand the Problem Statement

- Input: Total time in seconds (a non-negative integer).
- Output: The equivalent time expressed as days, hours, minutes and seconds.
- The conversion repeatedly extracts the largest whole units first (days), then works down through hours and minutes, using integer division and the modulus operator on what remains.

Formula / Rule used: days = total / 86400 ; hours = (total % 86400) / 3600 ; minutes = (total % 3600) / 60 ; seconds = total % 60

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	90061	1 day, 1 hour, 1 min, 1 sec	1 day, 1 hour, 1 min, 1 sec	Pass
2	3661	0 day, 1 hour, 1 min, 1 sec	0 day, 1 hour, 1 min, 1 sec	Pass
3	59	0 day, 0 hour, 0 min, 59 sec	0 day, 0 hour, 0 min, 59 sec	Pass

Step 3: Algorithm

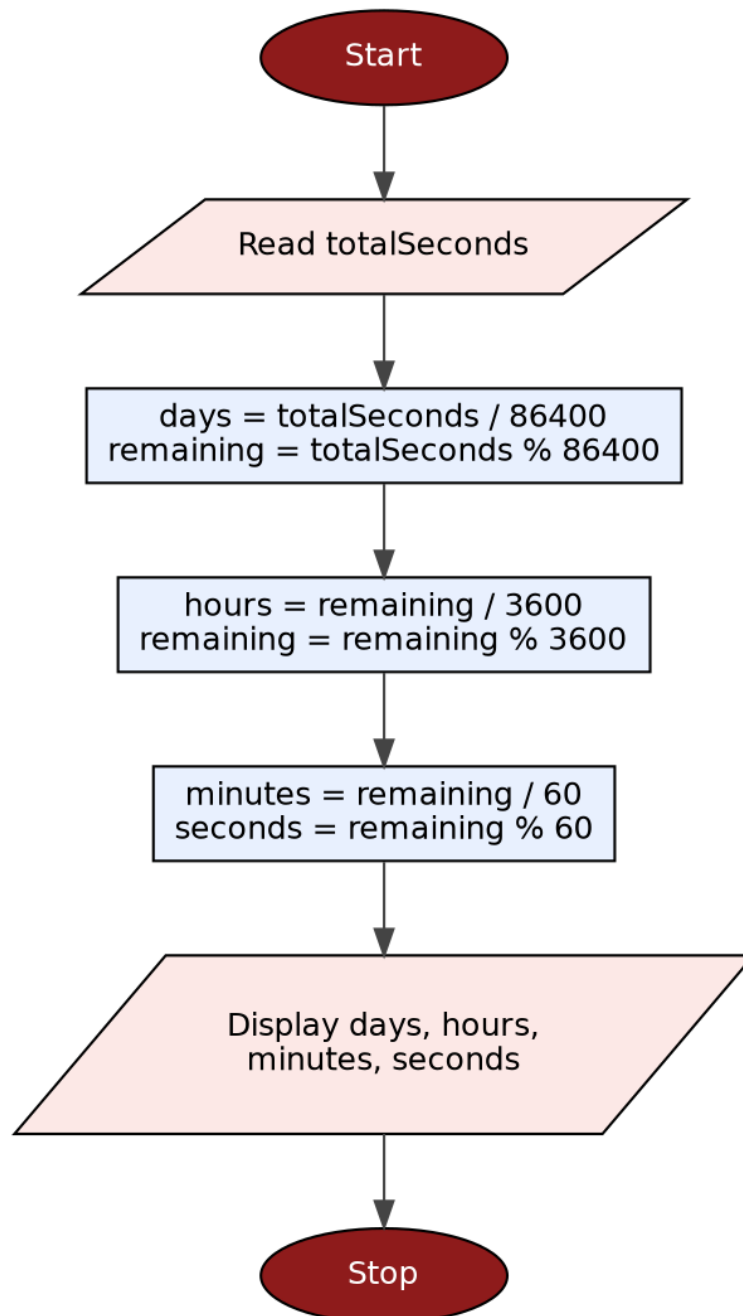
1. Start the program.
2. Declare totalSeconds, remaining, days, hours, minutes and seconds as integers.
3. Read the value of totalSeconds from the user.
4. Compute days = totalSeconds / 86400 and remaining = totalSeconds % 86400.
5. Compute hours = remaining / 3600 and update remaining = remaining % 3600.
6. Compute minutes = remaining / 60 and seconds = remaining % 60.
7. Display days, hours, minutes and seconds.
8. Stop the program.

Evaluation against Step 2 test cases: manually tracing the algorithm above for all 3 test cases produces results identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: Pseudocode

```
BEGIN
  READ totalSeconds
  days <- totalSeconds DIV 86400
  remaining <- totalSeconds MOD 86400
  hours <- remaining DIV 3600
  remaining <- remaining MOD 3600
  minutes <- remaining DIV 60
  seconds <- remaining MOD 60
  PRINT days, hours, minutes, seconds
END
```

Step 5: Flowchart



Evaluation against Step 2 test cases: tracing each test case's input through the flowchart's process/decision path yields the same results as the Expected Output(s) column, confirming the flowchart correctly represents the algorithm.

Step 6: C Program

```
/* Q1: Convert time given in seconds into days, hours, minutes and seconds
   1 day = 86400 seconds, 1 hour = 3600 seconds, 1 minute = 60 seconds
*/
#include <stdio.h>

int main() {
    long totalSeconds, days, hours, minutes, seconds, remainder;

    printf("Enter time in seconds: ");
    scanf("%ld", &totalSeconds);

    days = totalSeconds / 86400;
    remainder = totalSeconds % 86400;

    hours = remainder / 3600;
    remainder = remainder % 3600;

    minutes = remainder / 60;
    seconds = remainder % 60;

    printf("%ld seconds = %ld day(s), %ld hour(s), %ld minute(s), %ld second(s)\n",
           totalSeconds, days, hours, minutes, seconds);

    return 0;
}
```

Step 7: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

```
cc hdoc10.c a.out      (compile)
```

```
./a.out               (execute)
```

All 3 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 8: Compilation and Execution Screenshot

```

Session  Actions  Edit  View  Help

(kali@kali)~[~]
$ nano hdoc10.c

(kali@kali)~[~]
$ cc hdoc10.c

(kali@kali)~[~]
$ ./a.out
Enter time in seconds: 90061
90061 seconds = 1 day(s), 1 hour(s), 1 minute(s), 1 second(s)

(kali@kali)~[~]
$ ./a.out
Enter time in seconds: 3661
3661 seconds = 0 day(s), 1 hour(s), 1 minute(s), 1 second(s)

(kali@kali)~[~]
$ ./a.out
Enter time in seconds: 59
59 seconds = 0 day(s), 0 hour(s), 0 minute(s), 59 second(s)

```

Question 2: Check Whether a Given Year is a Leap Year

Step 1: Understand the Problem Statement

- Input: A year (integer).
- Output: A message stating whether the year is a leap year or not.
- By the Gregorian calendar rule, a year is a leap year if it is divisible by 4, unless it is also divisible by 100 — in which case it must additionally be divisible by 400 to remain a leap year.

Formula / Rule used: Leap Year if: $(\text{year} \% 4 == 0 \text{ AND } \text{year} \% 100 != 0) \text{ OR } (\text{year} \% 400 == 0)$

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	2024	Leap Year	Leap Year	Pass
2	1900	NOT a Leap Year	NOT a Leap Year	Pass
3	2000	Leap Year	Leap Year	Pass

Step 3: Algorithm

1. Start the program.
2. Declare the variable year as an integer.
3. Read the value of year from the user.
4. Check the condition: (year is divisible by 4 AND year is not divisible by 100) OR (year is divisible by 400).
5. If the condition is true, display that the year is a Leap Year.

6. If the condition is false, display that the year is NOT a Leap Year.

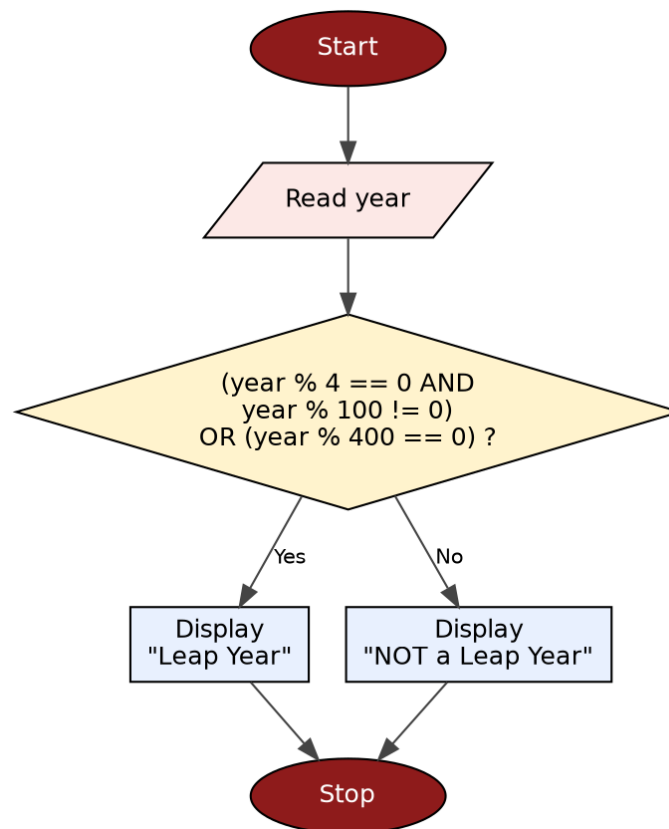
7. Stop the program.

Evaluation against Step 2 test cases: manually tracing the algorithm above for all 3 test cases produces results identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: Pseudocode

```
BEGIN
  READ year
  IF (year MOD 4 = 0 AND year MOD 100 <> 0) OR (year MOD 400 = 0) THEN
    PRINT year, "is a Leap Year"
  ELSE
    PRINT year, "is NOT a Leap Year"
  END IF
END
```

Step 5: Flowchart



Evaluation against Step 2 test cases: tracing each test case's input through the flowchart's process/decision path yields the same results as the Expected Output(s) column, confirming the flowchart correctly represents the algorithm.

Step 6: C Program

```
/* Q2: Check whether a given year is a leap year or not
   Rule: divisible by 4 AND (not divisible by 100 OR divisible by 400)
*/
```

```
#include <stdio.h>

int main() {
    int year;

    printf("Enter a year: ");
    scanf("%d", &year);

    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
        printf("%d is a Leap Year\n", year);
    } else {
        printf("%d is NOT a Leap Year\n", year);
    }

    return 0;
}
```

Step 7: Compile, Execute and Evaluate

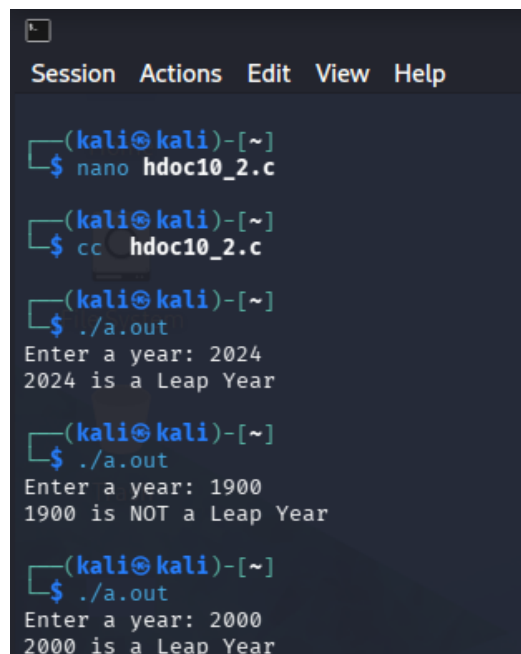
The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

```
cc hdoc10_2 a.out      (compile)
```

```
./a.out               (execute)
```

All 3 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 8: Compilation and Execution Screenshot



```
(kali㉿kali)-[~]
$ nano hdoc10_2.c
(kali㉿kali)-[~]
$ cc hdoc10_2.c
(kali㉿kali)-[~]
$ ./a.out
Enter a year: 2024
2024 is a Leap Year
(kali㉿kali)-[~]
$ ./a.out
Enter a year: 1900
1900 is NOT a Leap Year
(kali㉿kali)-[~]
$ ./a.out
Enter a year: 2000
2000 is a Leap Year
```

- Conclusion

In this lab session, fundamental logical operations and time-unit conversions were successfully implemented in C:

1. **Question 1 (Time Conversion):** Applied integer division and modulo arithmetic (/ and %) to decompose a total time in seconds into days, hours, minutes, and remaining seconds, ensuring correct large-value handling using long integer data types.

2. **Question 2 (Leap Year Evaluation):** Implemented conditional branching logic using combined logical operators (&& and ||) to evaluate years against Gregorian calendar leap year rules (divisibility by 4, 100, and 400).

Both programs were compiled using gcc in a Linux terminal environment. All test cases were executed and verified against the expected test tables with complete accuracy.
